Confidential DeFi: Private Trading, Sealed Auctions, and Encrypted Order Matching on Zoo Network

Eliminating MEV Through Fully Homomorphic Encryption

Version 2026.04

Zach Kelling

Zoo Labs Foundation

2026

Abstract

Decentralized finance leaks value at every layer. Front-running extracts $1.5B+ annually from Ethereum traders. Sandwich attacks inflate slippage. Information asymmetry between informed and uninformed participants produces a market where the fastest extractor wins, not the best price-discoverer. We present **Confidential DeFi** on Zoo Network: a suite of protocols where orders are encrypted at submission, matched homomorphically without decryption, and settled via threshold decryption of only the matched pairs. The core primitive is an FHE-native DEX virtual machine that evaluates encrypted limit orders using TFHE boolean circuits for price comparison and CKKS approximate arithmetic for volume matching. Sealed-bid auctions evaluate encrypted bids without revealing amounts until the auction concludes. A private AMM processes encrypted swap amounts, exposing only aggregate pool state to liquidity providers. All decryption is coordinated by Zoo T-Chain with a 67-of-100 validator quorum, ensuring no single party can observe orderflow. We benchmark the system at 340 encrypted order matches per second on GPU-accelerated validators, with end-to-end latency of 2.8 seconds from submission to settlement—practical for a blockchain DEX where block times are 2 seconds. Compared to existing solutions (Flashbots Protect, MEV-Share, Penumbra, Renegade), Zoo Confidential DeFi provides stronger privacy guarantees (full orderflow encryption vs. partial hiding) with competitive throughput.

Keywords: confidential DeFi, FHE, MEV, encrypted order matching, sealed-bid auction, private AMM, threshold decryption

Contents

1 Introduction

1.1 The MEV Crisis

Maximal Extractable Value (MEV) is the profit available to block producers by reordering, inserting, or censoring transactions within a block. On Ethereum, MEV extraction exceeded $1.5 billion in 2024 alone [1], borne entirely by ordinary users through:

1. **Front-Running.** A searcher observes a pending swap in the mempool, submits an identical swap with higher gas to execute first, and profits from the price impact.
2. **Sandwich Attacks.** A searcher brackets a victim's swap with a buy-before and sell-after, capturing the slippage as profit.
3. **Just-in-Time Liquidity.** A searcher provides liquidity immediately before a large swap (capturing fees) and removes it immediately after (avoiding impermanent loss).
4. **Time-Bandit Attacks.** Block producers reorg recent blocks to capture high-value MEV from past transactions.

These attacks share a common root cause: **transaction content is visible before execution.** The mempool is a public bulletin board. Anyone who can read it faster than others can extract value.

1.2 Why Existing Solutions Fall Short

Solution	Orderflow Privacy	On-Chain Matching	No Trusted Party
Flashbots Protect	Partial (relay sees orders)	No (off-chain relay)	No
MEV-Share	Partial (hints leaked)	No (off-chain)	No
Penumbra	Full (shielded pool)	No (MASP, not matching)	Yes
Renegade	Full (MPC matching)	No (off-chain MPC)	Partial
Zoo Confidential DeFi	**Full (FHE)**	**Yes**	**Yes**

Table 1: Comparison of MEV mitigation approaches

Flashbots and MEV-Share reduce MEV by hiding orderflow from searchers, but the relay operator still observes all transactions. Penumbra provides shielded transfers but does not support on-chain order matching. Renegade uses MPC for dark-pool matching but relies on off-chain computation with a limited participant set.

1.3 Our Approach

Zoo Confidential DeFi encrypts the entire orderflow lifecycle:

1. Orders are encrypted under a collective FHE public key before entering the mempool.
2. The DEX VM evaluates encrypted orders homomorphically—price comparisons, volume matching, and settlement calculations are performed on ciphertexts.
3. Only matched order pairs are decrypted via T-Chain threshold protocol.

4. Unmatched orders remain encrypted; no information is revealed about their content.

This eliminates MEV at the source: if no one can read the orders, no one can front-run them.

1.4 Paper Organization

Section 2 introduces the cryptographic primitives. Section 3 describes the encrypted orderflow protocol. Section 4 presents the FHE order matching engine. Section 5 covers sealed-bid auctions. Section 6 describes the private AMM. Section 7 details threshold settlement. Section 8 compares against existing solutions. Section 9 presents performance benchmarks. Section 10 analyzes security properties. Section 11 concludes.

2 Cryptographic Primitives

2.1 TFHE for Boolean Circuits

TFHE (Torus Fully Homomorphic Encryption) [2] operates on encrypted bits. Each ciphertext encrypts a single bit; boolean gates (AND, OR, NOT, XOR) are evaluated homomorphically with *programmable bootstrapping* to refresh noise. We use TFHE for:

- Price comparison: Is $\hat{p}_{bid} \geq \hat{p}_{ask}$? (encrypted comparison circuit)
- Auction winner determination: Which encrypted bid is highest?
- Order priority: Timestamp comparison for price-time priority matching

2.2 CKKS for Approximate Arithmetic

CKKS [3] encrypts vectors of approximate real numbers and supports addition and multiplication on ciphertexts. We use CKKS for:

- Volume matching: Computing fill quantities for partially matched orders
- AMM calculations: Constant-product invariant evaluation on encrypted amounts
- Fee computation: Calculating trading fees as a fraction of encrypted trade amounts

2.3 Collective Key Generation

The FHE public key is generated collectively by T-Chain validators using distributed key generation (DKG):

1. Each validator v_i generates a local key pair (sk_i, pk_i).
2. Validators run a DKG protocol to produce a collective public key $pk = \sum_{i=1}^{n} pk_i$.
3. The corresponding secret key sk is never materialized; it exists only as shares sk_i distributed across validators.
4. Decryption requires t-of-n validators (default $t = 67$, $n = 100$) to contribute partial decryptions.

Theorem 1 (Decryption Security)**.** *Under the RLWE assumption with parameters* ($N = 2^{14}, q = 2^{109}$)*, an adversary controlling fewer than t validators cannot decrypt any ciphertext with probability greater than* 2^{-128}*.*

3 Encrypted Orderflow

3.1 Order Structure

A limit order $O = (side, p, q, pair, \tau, did)$ consists of:

- $side \in \{\texttt{BUY}, \texttt{SELL}\}$: order direction
- $p \in \mathbb{R}^+$: limit price
- $q \in \mathbb{R}^+$: quantity
- *pair*: trading pair (e.g., KEEPER/USDC)
- τ: expiry timestamp
- *did*: submitter's decentralized identity

3.2 Encryption at Submission

Before entering the mempool, the submitter encrypts price and quantity:

$$\hat{p} = \text{TFHE.Encrypt}(pk, bits(p)) \tag{1}$$
$$\hat{q} = \text{CKKS.Encrypt}(pk, q) \tag{2}$$

Price is encrypted with TFHE (bit-level) because matching requires exact comparison. Quantity is encrypted with CKKS (approximate) because volume matching requires arithmetic.

The trading pair and expiry are left in plaintext—they are necessary for routing orders to the correct matching engine, and revealing them leaks no exploitable information.

3.3 Mempool Privacy

The encrypted order enters the mempool as:

$$\hat{O} = (side, \hat{p}, \hat{q}, pair, \tau, did, \sigma) \tag{3}$$

where σ is the submitter's signature over $\hat{O}$. Validators can verify the signature and check τ without decrypting price or quantity. The mempool reveals:

- That an order exists (unavoidable)
- Whether it is a buy or sell (needed for routing)
- Which trading pair (needed for routing)
- When it expires (needed for validity)

It does *not* reveal price or quantity—exactly the information needed for front-running and sandwich attacks.

3.4 Order Cancellation

A submitter can cancel an unmatched order by signing a cancellation message referencing the order's hash. The cancellation is processed without decryption; the order is simply removed from the matching engine.

4 FHE Order Matching

4.1 Price Comparison Circuit

The core matching operation is comparing an encrypted bid price $\hat{p}_b$ against an encrypted ask price $\hat{p}_a$. Using TFHE, we evaluate a greater-than-or-equal circuit on the bit representations:

Algorithm 1 Encrypted Price Comparison (TFHE)

Require: Encrypted bid price $\hat{p}_b = (\hat{b}_{63}, \ldots, \hat{b}_0)$, encrypted ask price $\hat{p}_a = (\hat{a}_{63}, \ldots, \hat{a}_0)$

$\hat{r} \leftarrow \text{TFHE.Encrypt}(pk, 0)$ {result bit}
for $i = 63$ **downto** 0 **do**
  $\hat{e}_i \leftarrow \text{TFHE.XNOR}(\hat{b}_i, \hat{a}_i)$ {bits equal?}
  $\hat{g}_i \leftarrow \text{TFHE.AND}(\hat{b}_i, \text{TFHE.NOT}(\hat{a}_i))$ {bid bit > ask bit?}
  $\hat{r} \leftarrow \text{TFHE.MUX}(\hat{e}_i, \hat{r}, \hat{g}_i)$ {propagate or update}
end for
return $\hat{r}$ {encrypted bit: 1 if $p_b \geq p_a$, 0 otherwise}

This circuit evaluates in $O(b)$ where $b = 64$ is the bit width of the price representation. With programmable bootstrapping after each gate, noise does not accumulate.

4.2 Volume Matching

When a price match is found ($\hat{r} = \hat{1}$), the fill quantity is computed via CKKS:

$$\hat{q}_{fill} = \text{CKKS.Min}(\hat{q}_b, \hat{q}_a) \tag{4}$$

where CKKS.Min is implemented using the sign function approximated by a Chebyshev polynomial of degree 16:

$$\text{CKKS.Min}(\hat{x}, \hat{y}) = \frac{\hat{x} + \hat{y} - |\hat{x} - \hat{y}|}{2} \tag{5}$$

with $|\hat{z}| \approx \hat{z} \cdot \text{sign}(\hat{z})$ evaluated homomorphically.

4.3 Matching Engine Architecture

The matching engine processes orders in price-time priority:

Remark 1. The naive $O(|\mathcal{B}| \cdot |\mathcal{A}|)$ matching is acceptable because: (a) each comparison is independent and parallelizable across GPU cores, and (b) active order book depth on Zoo DEX is bounded (typically $< 1{,}000$ orders per pair).

Algorithm 2 FHE Order Matching Engine

Require: Encrypted order books $\hat{\mathcal{B}}$ (bids), $\hat{\mathcal{A}}$ (asks), sorted by submission time

$matches \leftarrow \emptyset$
for each bid $\hat{O}_b \in \hat{\mathcal{B}}$ **do**
  for each ask $\hat{O}_a \in \hat{\mathcal{A}}$ **do**
    $\hat{r} \leftarrow \text{TFHE.GEQ}(\hat{p}_b, \hat{p}_a)$ {price match?}
    $\hat{q}_f \leftarrow \text{CKKS.Min}(\hat{q}_b, \hat{q}_a)$ {fill quantity}
    $\hat{m} \leftarrow \text{TFHE.AND}(\hat{r}, \text{TFHE.GT}(\hat{q}_f, \hat{0}))$ {valid match?}
    Append $(\hat{O}_b, \hat{O}_a, \hat{q}_f, \hat{m})$ to *matches*
    if $\hat{m}$ decrypts to 1 (via threshold) **then**
      Update remaining quantities: $\hat{q}_b \leftarrow \hat{q}_b - \hat{q}_f$, $\hat{q}_a \leftarrow \hat{q}_a - \hat{q}_f$
    end if
  end for
end for
return *matches*

4.4 Batch Matching Optimization

Rather than matching orders one-at-a-time, the engine batches all orders received within a block period (2 seconds) and matches them simultaneously. This provides:

- **Fairness.** All orders in a batch are treated equally—no time advantage within a block.
- **Efficiency.** GPU parallelism is maximized when processing a batch.
- **MEV elimination.** Reordering within a batch does not affect matching outcomes when price-time priority considers only batch ordering.

5 Sealed-Bid Auctions

5.1 Auction Structure

A sealed-bid auction on Zoo operates as follows:

1. **Commitment Phase.** Bidders encrypt their bids under the collective FHE key and submit them on-chain. The auction contract records encrypted bids but cannot read them.
2. **Evaluation Phase.** After the bidding deadline, the auction contract evaluates encrypted bids homomorphically to determine the winner.
3. **Reveal Phase.** Only the winning bid (and, in second-price auctions, the second-highest bid) is decrypted via T-Chain threshold protocol. Losing bids are never decrypted.

5.2 TFHE Winner Determination

For a first-price auction with n encrypted bids $\hat{b}_1, \ldots, \hat{b}_n$:

Algorithm 3 Encrypted First-Price Auction

Require: Encrypted bids $\hat{b}_1, \ldots, \hat{b}_n$

$\hat{w} \leftarrow \hat{b}_1$ {current winner}
$\hat{j} \leftarrow \text{TFHE.Encrypt}(pk, 1)$ {winner index}
for $i = 2$ to n **do**
  $\hat{r} \leftarrow \text{TFHE.GT}(\hat{b}_i, \hat{w})$
  $\hat{w} \leftarrow \text{TFHE.MUX}(\hat{r}, \hat{b}_i, \hat{w})$
  $\hat{j} \leftarrow \text{TFHE.MUX}(\hat{r}, \text{TFHE.Encrypt}(pk, i), \hat{j})$
end for
Submit $(\hat{w}, \hat{j})$ to T-Chain for threshold decryption
return Winner index j, winning bid w

5.3 Second-Price (Vickrey) Auction

In a Vickrey auction, the winner pays the second-highest bid. The circuit extends the above to track both the highest and second-highest encrypted values:

$$(\hat{w}_1, \hat{w}_2) = \text{TFHE.TopTwo}(\hat{b}_1, \ldots, \hat{b}_n) \tag{6}$$

The winner is determined by $\hat{w}_1$; the settlement price is $\hat{w}_2$. Only $\hat{w}_2$ is decrypted for settlement. The actual winning bid amount is never revealed—a property impossible in traditional auctions.

5.4 Auction Applications

- **NFT Auctions.** Sealed-bid NFT sales without sniping or bid-shading.
- **Block Space Auctions.** Validators auction inclusion rights; builders submit encrypted bids for block space.
- **Token Launch Auctions.** Fair price discovery for new tokens without front-running.
- **Liquidation Auctions.** DeFi protocol liquidations without predatory bidding.

6 Private AMM

6.1 Encrypted Constant-Product Invariant

A standard constant-product AMM maintains the invariant $x \cdot y = k$ where x and y are reserve amounts. In the private AMM, reserves are encrypted:

$$\text{CKKS.Mul}(\hat{x}, \hat{y}) = \hat{k} \tag{7}$$

A swap of encrypted amount $\hat{\Delta x}$ for token X yields:

$$\hat{\Delta y} = \hat{y} - \frac{\hat{k}}{\hat{x} + \hat{\Delta x}} \tag{8}$$

This computation is evaluated entirely in CKKS, producing an encrypted output amount $\hat{\Delta y}$ that is decrypted only for the swapper via T-Chain.

6.2 Liquidity Provider View

Liquidity providers cannot see individual swap amounts. They observe only:

- Aggregate pool reserves (decrypted periodically, e.g., every 100 blocks)
- Total fees earned (decrypted for LP withdrawal)
- Pool utilization metrics (computed homomorphically)

This protects swappers from LP-side information extraction while giving LPs sufficient information for position management.

6.3 Concentrated Liquidity

Encrypted concentrated liquidity (a la Uniswap V3) requires range-bound position management:

1. LPs define price ranges in plaintext (the range is public; the amount is private).
2. Deposit amounts are encrypted: $\hat{L} = \text{CKKS.Encrypt}(pk, L)$.
3. The AMM uses TFHE to check whether the current price falls within each LP's range.
4. Active positions contribute encrypted liquidity; out-of-range positions contribute nothing.

6.4 Impermanent Loss Protection

Because swap amounts are encrypted, LPs cannot be targeted by toxic flow specifically designed to maximize impermanent loss. An attacker cannot observe pool state in real-time and craft adversarial trades.

7 Threshold Settlement

7.1 T-Chain Coordination

Settlement is the only phase where decryption occurs. T-Chain coordinates:

1. The matching engine submits matched pairs $(\hat{O}_b, \hat{O}_a, \hat{q}_f)$ to T-Chain.
2. T-Chain validators verify the match proof (the encrypted comparison result $\hat{r} = \hat{1}$).
3. Each validator computes a partial decryption of $\hat{q}_f$ using its key share sk_i.
4. Once $t = 67$ partial decryptions are collected, the fill quantity q_f is reconstructed.
5. The settlement contract transfers q_f tokens from seller to buyer and $q_f \cdot p_{match}$ payment from buyer to seller.

7.2 Partial Revelation

The settlement reveals only the information necessary for execution:

Unmatched orders are *never* decrypted. They expire silently, revealing nothing about the submitter's valuation.

Information	Matched Orders	Unmatched Orders
Existence	Revealed	Revealed
Side (buy/sell)	Revealed	Revealed
Price	**Match price only**	**Never revealed**
Quantity	**Fill quantity only**	**Never revealed**
Submitter identity	Revealed to counterparty	Never revealed

Table 2: Information revelation under Confidential DeFi

7.3 Atomic Settlement

Settlement is atomic: either both legs of the trade execute or neither does. This is enforced by the settlement contract on Zoo EVM:

```
function settle(
    bytes32 match_id,
    uint256 fill_qty,
    uint256 match_price,
    bytes   threshold_proof
) external {
    require(TChain.verify(threshold_proof, match_id));

    token_base.transferFrom(seller, buyer, fill_qty);
    token_quote.transferFrom(buyer, seller,
                             fill_qty * match_price);

    emit Settlement(match_id, fill_qty, match_price);
}
```

8 Comparison to Existing Solutions

8.1 Flashbots Protect

Flashbots Protect routes transactions through a private relay, hiding them from the public mempool. The relay operator (Flashbots) observes all transactions. Users must trust the relay not to extract MEV itself. Zoo Confidential DeFi requires no trusted relay; orders are encrypted under a collective key that no single party holds.

8.2 MEV-Share

MEV-Share allows users to share partial information ("hints") about their transactions with searchers, receiving a portion of extracted MEV in return. This reduces but does not eliminate MEV—users still lose value, just less of it. Zoo eliminates MEV entirely: there is no information to share because there is no information to extract.

8.3 Penumbra

Penumbra implements a shielded pool (Multi-Asset Shielded Pool, MASP) for private transfers and swaps. Swaps use a batch auction mechanism where all swaps in a block execute at the

same clearing price. However, Penumbra does not support limit orders, on-chain order books, or sealed-bid auctions. Zoo provides full order book functionality with encrypted limit orders.

8.4 Renegade

Renegade implements a dark pool using multiparty computation (MPC). Two-party matching is performed via garbled circuits. Limitations: (a) matching is pairwise (two parties at a time), not batch; (b) both parties must be online simultaneously; (c) the MPC coordinator observes metadata. Zoo's FHE-based approach handles arbitrary batch sizes, operates asynchronously, and reveals no metadata to validators.

8.5 Summary

Property	**Flashbots**	**MEV-Share**	**Penumbra**	**Renegade**	**Zoo**
Full orderflow privacy	No	No	Yes	Yes	**Yes**
On-chain matching	No	No	Batch only	No	**Yes**
Limit orders	N/A	N/A	No	Yes	**Yes**
Sealed auctions	No	No	No	No	**Yes**
No trusted party	No	No	Yes	Partial	**Yes**
Asynchronous	Yes	Yes	Yes	No	**Yes**

Table 3: Feature comparison across MEV mitigation approaches

9 Benchmarks

All benchmarks measured on Zoo Network testnet with 100 validators, each running AMD EPYC 7763 with 256 GB RAM and NVIDIA A100 80 GB.

9.1 Encrypted Order Operations

Operation	**Latency (GPU)**	**Throughput**
Order encryption (TFHE price + CKKS qty)	1.2 ms	833 orders/s
Price comparison (64-bit TFHE)	4.1 ms	243 comparisons/s
Volume matching (CKKS min)	0.8 ms	1,250 matches/s
Full match (compare + volume)	4.9 ms	204 matches/s
Batch match (100 orders)	1.44 s	340 matches/s

Table 4: Encrypted order operation benchmarks

9.2 Sealed-Bid Auction

Linear scaling: each additional bid requires one encrypted comparison (4.1 ms on GPU).

Auction Size	Evaluation Time	Per-Bid Cost
10 bids	41 ms	4.1 ms
100 bids	410 ms	4.1 ms
1,000 bids	4.1 s	4.1 ms
10,000 bids	41 s	4.1 ms

Table 5: Sealed-bid auction evaluation (first-price, GPU-accelerated)

Operation	Latency	Notes
Encrypted swap (constant product)	3.2 ms	CKKS arithmetic
Encrypted swap (concentrated)	8.7 ms	Range check + CKKS
Pool state aggregation (per 100 blocks)	124 ms	Threshold decrypt
LP fee calculation	1.1 ms	CKKS multiplication

Table 6: Private AMM operation benchmarks

9.3 Private AMM

9.4 Threshold Settlement

The 2.8-second end-to-end latency is dominated by the 2.0-second block confirmation time. Cryptographic overhead (matching + threshold decryption) adds only 0.8 seconds.

9.5 Comparison to Plaintext DEX

The computational overhead is significant (490× for a single match) but is amortized by batch processing and GPU acceleration. The economic overhead is negative: users save more from eliminated MEV than they pay in additional gas.

10 Security Analysis

10.1 Threat Model

We consider an adversary who:

1. Controls up to $t - 1 = 66$ of 100 T-Chain validators.
2. Observes all network traffic (encrypted mempool, block contents).
3. Can submit arbitrary orders (including adversarial ones).
4. Has unbounded computational power *except* for breaking RLWE or LWE.

10.2 Properties

Theorem 2 (Orderflow Privacy)**.** *Under the RLWE assumption, an adversary controlling fewer than t validators learns nothing about the price or quantity of any order beyond what is revealed by the match outcome.*

Phase	Latency	Bottleneck
Match proof verification	12 ms	Computation
Partial decryption (per validator)	0.8 ms	Computation
Quorum collection (67-of-100)	240 ms	Network RTT
Settlement transaction	2.0 s	Block confirmation
Total (submission to settlement)	**2.8 s**	Block time

Table 7: End-to-end settlement latency

Metric	Plaintext DEX	Zoo Confidential	Overhead
Order submission	0.1 ms	1.2 ms	12×
Single match	0.01 ms	4.9 ms	490×
Batch match (100)	0.8 ms	1.44 s	1,800×
Settlement	2.0 s	2.8 s	1.4×
MEV extracted	\$1.5B/yr	\$0	$-\infty$

Table 8: Confidential DeFi overhead vs. plaintext execution

Proof sketch. Order prices and quantities are encrypted under the collective FHE key. Decryption requires t key shares. An adversary with $< t$ shares cannot decrypt (information-theoretic for Shamir sharing with $< t$ shares). The matching engine operates on ciphertexts; intermediate values are also encrypted. Only matched pairs are decrypted, revealing only the fill price and quantity. □

Theorem 3 (MEV Elimination)**.** *If orderflow privacy holds, then for any block producer strategy σ, the MEV extractable from reordering, inserting, or censoring encrypted orders is zero.*

Proof sketch. MEV extraction requires observing order content (price, quantity) before execution. Under orderflow privacy, this content is hidden. Reordering encrypted orders of the same side has no effect on batch matching. Inserting orders without knowledge of existing order prices cannot systematically extract value. Censoring orders is detectable (missing from the block) and punishable via slashing. □

10.3 Side-Channel Mitigation

Potential information leakage:

- **Order count per block.** Mitigated by padding: each block contains a fixed number of order slots, with dummy encrypted orders filling empty slots.
- **Ciphertext size.** Uniform: all encrypted orders have identical byte length regardless of price/quantity magnitude.
- **Timing.** Batch processing ensures all orders in a block are matched simultaneously; no timing side-channel from sequential processing.

11 Conclusion

DeFi's transparency is both its strength and its fatal flaw. The same public mempool that enables permissionless participation also enables systematic value extraction. MEV is not a bug in DeFi—it is a consequence of plaintext orderflow on a public ledger.

Confidential DeFi on Zoo Network eliminates this problem at the cryptographic level. Orders are encrypted before they enter the mempool. Matching happens on ciphertexts. Only matched pairs are decrypted. The result is a trading system where the fastest extractor has no advantage because there is nothing to extract.

The overhead is real: 490× for a single encrypted match compared to plaintext. But this comparison misses the point. The relevant comparison is not against a platonic frictionless market; it is against the actual DeFi market where users lose $1.5 billion annually to MEV. Against that baseline, a 2.8-second settlement with zero MEV is an improvement.

Three open problems remain:

1. **Cross-Chain Confidential DeFi.** Extending encrypted orderflow to atomic swaps across chains, where each chain holds a different FHE key.

2. **Encrypted Derivatives.** Applying FHE to options and futures, where payoff calculations are more complex than spot matching.

3. **Recursive FHE.** Using recursive bootstrapping to support unbounded-depth circuits, enabling complex matching algorithms (e.g., pro-rata allocation) without noise budget exhaustion.

Front-running dies when there is nothing to see.

References

[1] Flashbots, "MEV-Explore: Quantifying Maximal Extractable Value," `https://explore.flashbots.net`, 2024.

[2] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast Fully Homomorphic Encryption over the Torus," *J. Cryptology*, vol. 33, pp. 34–91, 2020.

[3] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," *Proc. ASIACRYPT*, Springer, 2017.

[4] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *Proc. IEEE S&P*, 2020.

[5] Z. Kelling, "Zoo DEX: Decentralized Exchange on Zoo Network," Zoo Labs Foundation, 2026.

[6] Z. Kelling, "Threshold Fully Homomorphic Encryption on Zoo Network: GPU-Accelerated Confidential Computation," Zoo Labs Foundation, 2026.

[7] Z. Kelling, "Threshold Signatures on Zoo T-Chain," Zoo Labs Foundation, 2026.

[8] Penumbra Labs, "Penumbra Protocol Specification," `https://protocol.penumbra.zone`, 2023.

[9] Renegade, "Renegade: On-Chain Dark Pool Protocol," `https://renegade.fi/whitepaper.pdf`, 2023.

[10] W. Vickrey, "Counterspeculation, Auctions, and Competitive Sealed Tenders," *J. Finance*, vol. 16, no. 1, pp. 8–37, 1961.